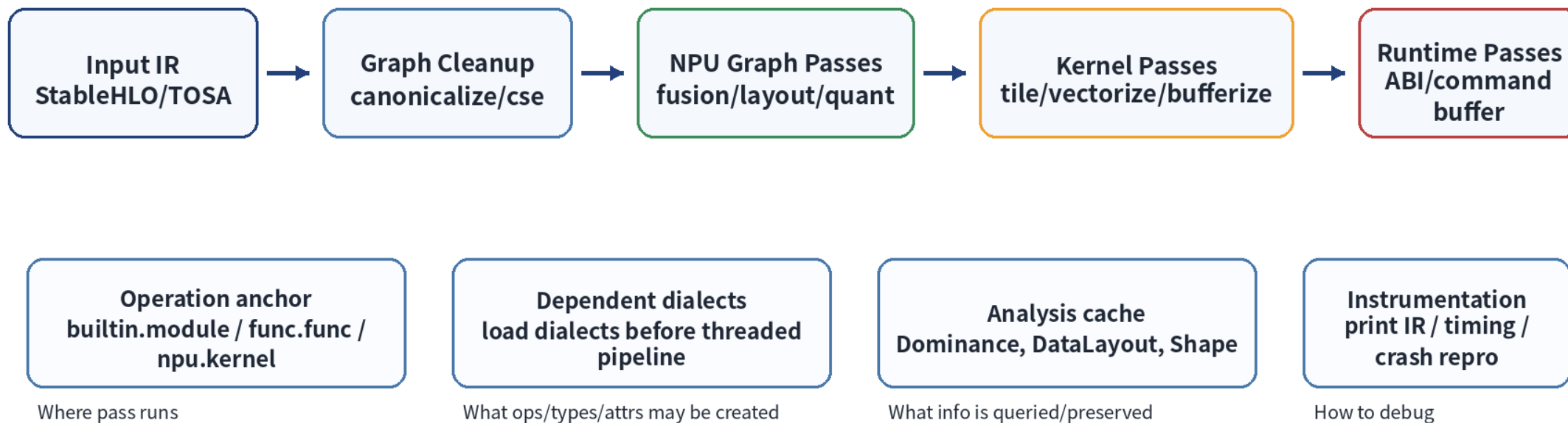


Lecture 09 - Pass Infrastructure

NPU Lowering Pass Pipeline Draft

Pass Infrastructure: NPU Lowering Pipeline



오늘의 위치

- 1-8 강 : IR, Dialect, StableHLO/TOSA, Linalg, SCF/Affine
- 9 강 : 실제 compiler 를 움직이는 pass infrastructure
- 10-15 강 : Rewrite, Conversion, Bufferization, Vector/Transform 으로 확장

Learning Objectives

- MLIR pass 의 실행 단위와 제약을 설명한다
- operation anchor 와 nested pass pipeline 을 읽고 작성한다
- dependent dialect, analysis, failure, instrumentation 개념을 NPU compiler 에 연결한다
- MatMul+Bias+ReLU 용 NPU lowering pipeline 초안을 작성한다

Pass 가 필요한 이유

- IR 은 data structure 이고 , pass 는 IR 을 변화시키는 compiler action
 - 최적화 , 정규화 , lowering, 검증 , instrumentation 을 작게 쪼갠 단위
- NPU compiler 에서는 fusion/layout/tile/buffer/schedule/emit 을 별도 pass 로 분리해야 debug 와 regression 이 가능

Pass Infrastructure 한 장 요약

- Pass = current operation 을 읽고 하위 IR 을 변환하는 unit
- PassManager = pass ordering, nesting, threading, instrumentation 을 관리
- Analysis = lazy/cached compiler fact
- Pipeline = command line 과 code 에서 재현 가능한 compiler recipe

공식 문서 관점

- MLIR pass 는 transformation/optimization 의 기본 infrastructure
- OperationPass 가 기본 단위이며 pass manager 는 operation nesting level 별로 pass 를 schedule
- mlir-opt 는 textual IR 을 parse 하고 pass sequence 를 실행한 뒤 다시 serialize 하는 test/debug utility

Pass = Current Operation Contract

- Pass 는 current operation 과 그 하위 operation 을 대상으로 동작
- sibling operation state 를 가정하거나 수정하면 multi-threaded execution 에서 위험
- pass instance 는 여러 operation 에 복사되어 실행될 수 있으므로 mutable global/pass state 금지
- NPU pass 는 module/function/kernel 중 정확한 anchor 를 선택해야 함

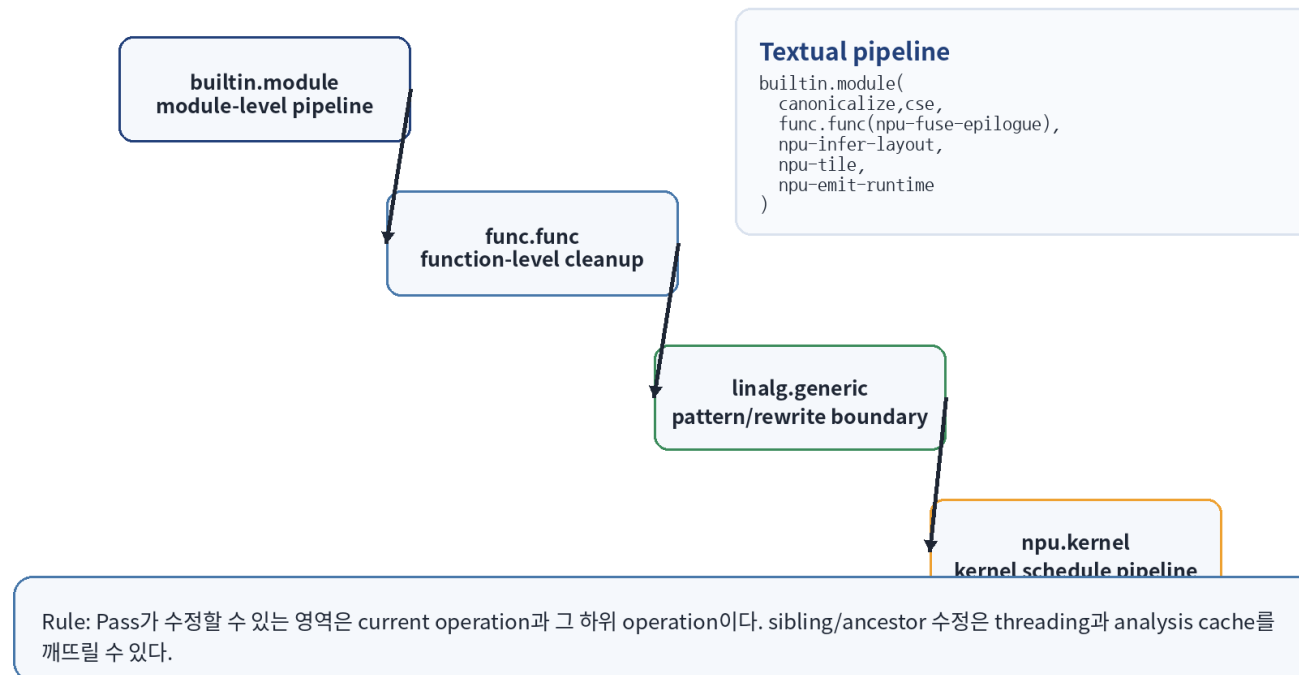
OperationPass 종류

- `OperationPass<>`: op-agnostic pass
- `OperationPass<ModuleOp>`: module-level pass
- `OperationPass<func::FuncOp>`: function-level pass
- `InterfacePass<FunctionOpInterface>`: interface 기반 filtered pass
- NPU 예 : `npu.kernel` 전용 scheduling pass

Operation Anchoring

- builtin.module: symbol table, whole-graph legality, ABI emission
- func.func: linalg/tensor graph cleanup, fusion, local optimization
- npu.kernel: tile loop, DMA/barrier, SRAM allocation
- any(...): 여러 viable isolated op 에 generic pass 적용

Operation Anchoring and Nested Pass Managers



Nested Pass Pipeline

- Textual pipeline 은 `op-anchor(pass, nested-pipeline, options)` 형태
 - 예 : `builtin.module(func.func(cse,canonicalize), npu-infer-layout)`
- nested manager 는 IR nesting 구조에 맞춰 실행
- NPU 에서는 function pass 와 kernel pass 를 섞을 때 nesting boundary 가 중요

Pass Ordering 원칙

- Canonicalize/CSE 는 transformation 사이의 glue pass
- Legality 가 높은 IR 에서 fusion/layout 을 먼저 결정
- Bufferization 이후에는 tensor-level fusion 자유도가 줄어듦
- Memory-space assignment 는 tile size 와 DMA schedule 전에 결정되어야 함
- Runtime ABI emission 은 마지막 boundary

NPU Compiler Pass Taxonomy

- Cleanup pass: canonicalize, cse, shape cleanup
- Graph pass: op legalization, fusion, layout propagation, quant policy
- Kernel pass: tiling, vectorization, memory-space assignment
- Schedule pass: DMA/barrier insertion, bank-conflict fix
- Runtime pass: descriptor/command buffer emission

Pass Lifecycle

- Register -> Configure -> Initialize -> Run -> Verify -> Preserve
- TableGen Pass spec 로 boilerplate 를 줄임
- runOnOperation() 에서 IR mutation 수행
- signalPassFailure() 는 pipeline 을 중단시키는 정상 실패 경로

Anatomy of a Pass Execution



```
struct FuseMatmulEpiloguePass : PassWrapper<..., OperationPass<func::FuncOp>> {
  void getDependentDialects(DialectRegistry &r) const override { r.insert<npu::NpuGraphDialect>(); }
  void runOnOperation() override {
    func::FuncOp f = getOperation();
    if (failed(matchAndRewriteMatmulBiasRelu(f))) return signalPassFailure();
    markAnalysesPreserved<DominanceInfo>(); // only if still valid
  }
};
```

C++ Pass Skeleton

```
struct FuseMatmulEpiloguePass
: PassWrapper<..., OperationPass<func::FuncOp>> {
void getDependentDialects(DialectRegistry &r) const override {
    r.insert<npu::NpuGraphDialect>();
}
void runOnOperation() override {
    func::FuncOp f = getOperation();
    if (failed(matchAndRewrite(f))) {
        f.emitError("illegal NPU epilogue pattern");
        return signalPassFailure();
    }
    ++numFused;
}
};
```

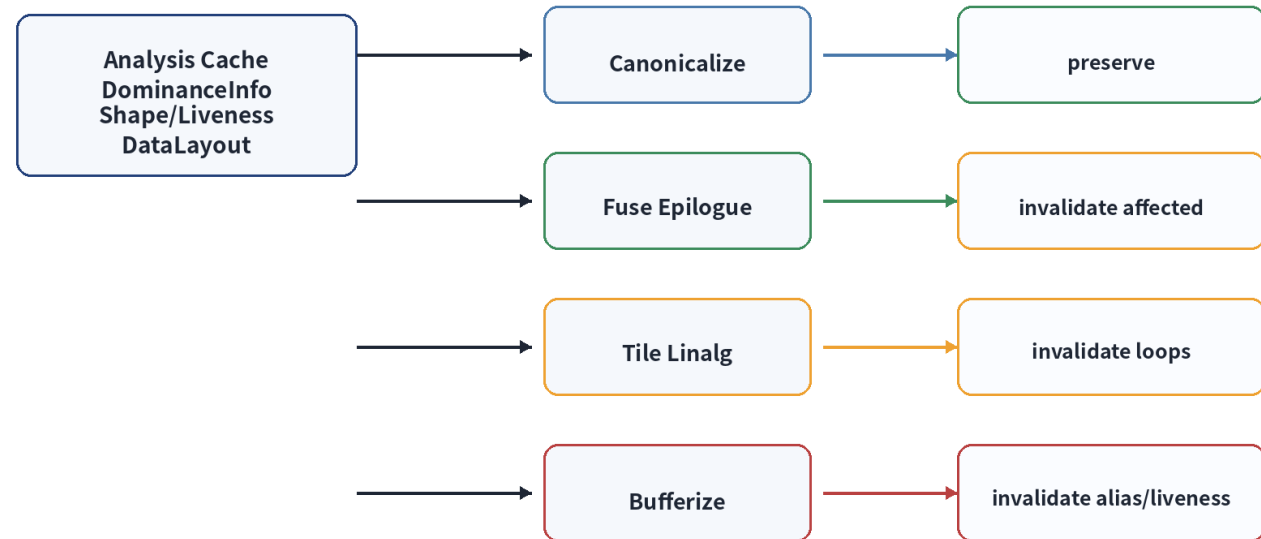
Dependent Dialects

- Pass 가 새 operation/type/attribute 를 만들 수 있으면 dialect 를 명시
- multi-threaded pass pipeline 실행 전 dialect load 가 필요
 - 예 : Linalg -> NPU pass 는 npu_graph/npu_kernel dialect 를 dependent 로 등록
- 입력 IR 에 이미 존재하는 dialect 만 읽는 것은 보통 dependent 등록 대상이 아님

Analysis Management

- Analysis 는 IR fact 를 계산하고 cache 하는 독립 class
- Transformation pass 가 IR 을 바꾸면 관련 analysis 가 invalidated
 - 변경하지 않았거나 invariant 가 유지되면 markAnalysesPreserved 사용
- NPU 예 : shape info, layout feasibility, SRAM footprint, liveness, dominance

Analysis and Preservation: Compiler Performance Contract



NPU tip: expensive analyses such as layout feasibility, SRAM footprint, and bank-conflict checks should be cached, but only preserved when t

Preservation 예시

- CSE 가 순수 op 만 제거 : dominance 는 유지될 수 있음
- Fusion 이 producer/consumer region 을 합침 : use-def/liveness 재계산 필요
- Tiling 이 loop nest 를 새로 만듦 : loop/dominance 관련 analysis 주의
- Bufferization 은 tensor alias/lifetime 가정을 크게 바꾸므로 보수적으로 invalidation

Pass Options

- Pass option 은 compile-time heuristic 을 재현 가능하게 만든다
 - 예 : tile-m=64, tile-n=128, tile-k=32
 - 예 : target=sram_512kb, acc=i32, enable-double-buffer=true
- NPU tuning knob 은 pass option 과 pipeline option 으로 노출

Pass Statistics

- 몇 개의 op 를 fuse 했는지 , tile 을 몇 개 만들었는지 , spill 이 몇 번 발생했는지 기록
- NPU compiler CI 에서 performance regression signal 로 활용
- 예 : numFusedEpilogues, numSramSpills, numDmaOps, numBankConflictFixes

Declarative Pass Spec

- TableGen Pass spec 는 summary, description, dependentDialects, options, statistics 를 선언
 - 생성된 base class 를 상속하면 반복 boilerplate 감소
- Pass 문서 자동 생성과 command-line option 정합성에 유리
 - 산업용 NPU compiler 에서는 pass registry 와 문서 자동화가 중요

Pass Registration

- PassRegistration<MyPass>() 로 pass 를 command line 에서 사용할 수 있게 등록
- PassPipelineRegistration 으로 복합 pipeline 을 하나의 이름으로 등록
 - 예 : --npu-lowering-pipeline{target=npu256,sram-kb=512}
- tool, test, runtime compiler 가 동일 pipeline string 을 공유해야 재현성 확보

Dynamic Pass Pipeline

- Pass 내부에서 조건부로 nested pipeline 실행 가능
- `runPipeline(OpPassManager &, Operation *)` 사용
 - 예 : `npu.partition` 이 각 subgraph 의 shape/layout 에 따라 다른 kernel pipeline 을 실행
 - 단 , 일반적으로 Pass 내부에서 별도 PassManager 를 직접 만들지 않는 편이 안전

Instrumentation

- IR printing: before/after/change/failure
- Timing: pass 별 compile-time hotspot 분석
- Statistics: transformation count 추적
- Crash reproducer: 실패 재현 가능한 MLIR snapshot 생성
- NPU compiler bring-up 에서는 필수 debug interface

Debug Flags Recipe

```
mlir-opt input.mlir \  
  --pass-pipeline='builtin.module(func.func(cse,canonicalize),npu-lowering-pipeline)' \  
  --mlir-print-ir-after-change \  
  --mlir-print-ir-after-failure \  
  --mlir-timing \  
  --mlir-pass-statistics \  
  --mlir-pass-pipeline-crash-reproducer=npu_fail.mlir
```


NPU Lowering Pipeline Draft

- Import -> Cleanup -> Legalize -> Fuse -> Layout -> Tile -> Bufferize -> MemorySpace -> DMA/Barrier -> Runtime
- 각 단계마다 input/output dialect 와 correctness check 를 명시
- 9 강 실습의 최종 산출물

NPU Lowering Pass Pipeline Draft

Stage	Representative passes	IR boundary	Correctness check
Import	legalize-stablehlo/tosa, canonicalize	Tensor IR	shape/dtype preserved
Graph	fuse-matmul-epilogue, infer-layout	npu_graph	legal op set
Kernel	tile-linalg, vectorize, assign-memory-space	npu_kernel	SRAM fits
Memory	one-shot-bufferize, buffer-deallocation	memref	alias/lifetime valid
Schedule	insert-dma, place-barriers, bank-conflict-fix	SCF/Affine/NPU	dependency valid
Runtime	emit-descriptors, emit-command-buffer	npu_rt	ABI stable

Draft deliverable

각 pass마다 입력 dialect, 출력 dialect, 생성할 dependent dialect, 필요한 analysis, invalidation/preservation, FileCheck test를 명시한다.

Pipeline Stage 1: Import/Cleanup

- Input: StableHLO/TOSA 또는 framework graph dialect
- Pass: shape canonicalization, constant folding, canonicalize, cse
- Output: tensor-level IR
- Check: dtype/shape semantics preserved

Pipeline Stage 2: Graph Legalization/Fusion

- Pass: decompose unsupported op, fold broadcast, fuse matmul+bias+activation
- Output: npu_graph candidate op
- Check: target legal op set, quantization metadata, layout contract
- Failure path: unsupported dynamic shape or dtype emits diagnostic

Pipeline Stage 3: Kernelization

- Pass: tile M/N/K, vectorize, choose accumulator type, allocate scratch buffers
- Output: npu_kernel ops or SCF/Affine loop nest
- Check: SRAM/accumulator footprint, edge tile policy, bank alignment

Pipeline Stage 4: Scheduling

- Pass: insert DMA load/store, double-buffering, barrier/await placement
- Check: producer-consumer dependency and memory hazard absence
- NPU target knobs: DMA channels, SRAM banks, outstanding transactions

Pipeline Stage 5: Runtime/ABI

- Pass: tensor descriptor, command buffer, kernel metadata emission
- Output: `npu_rt.launch` or binary command section
- Check: ABI version, alignment, descriptor field coverage, host/runtime compatibility

MatMul+Bias+ReLU Case Study

- Start: `linalg.matmul + arith.addf + arith.maximumf`
- Fuse epilogue while tensor semantics are still high-level
- Tile after fusion to avoid separate write/read of C
- Bufferize after layout/tile decisions
- Emit `npu.matmul` with `epilogue=relu` or command sequence

실습 A - Pipeline 읽기

- 제공된 01_pass_pipeline_input.mlir 읽기
- mlir-opt pipeline string 에서 anchor 와 pass order 표시
- 각 pass 가 생성할 수 있는 dialect 와 필요한 analysis 를 표로 작성

실습 B - Pass Skeleton 채우기

- 03_npu_pass_skeleton.cpp 의 TODO 채우기
- dependent dialect 추가
- getOperation(), walk/match, signalPassFailure 위치 표시
- 통계 counter 와 option 이름 작성

실습 C - NPU Lowering Pipeline 초안

- MatMul+Bias+ReLU 기준으로 8-12 개 pass 를 순서대로 배치
- 각 pass 별 input dialect, output dialect, legality check, FileCheck test 작성
- 성능 목표 : 중간 DRAM round-trip 제거 , SRAM footprint 제한 , compile-time debug 가능성

Common Mistakes

- Pass 가 sibling/parent IR 을 임의 수정
- dependent dialect 누락
- analysis preservation 을 과감하게 선언
- pass option 없이 hidden heuristic 사용
- bufferization 이후에 graph fusion 을 시도
- FileCheck 가 너무 brittle 하여 SSA 이름 변경에 취약

Takeaways

- Pass 는 NPU compiler 의 product-quality boundary
 - 좋은 pipeline 은 IR legality, analysis, debug, test 가 함께 설계된다
- Pass ordering 은 성능과 correctness 를 동시에 결정한다
 - 다음 강의 : Rewrite Pattern 으로 실제 fusion/transformation 구현

References

MLIR Pass Infrastructure

<https://mlir.llvm.org/docs/PassManagement/>

MLIR Using mlir-opt

<https://mlir.llvm.org/docs/Tutorials/MlirOpt/>

MLIR Passes

<https://mlir.llvm.org/docs/Passes/>

MLIR Language Reference

<https://mlir.llvm.org/docs/LangRef/>

MLIR Quickstart Rewrites

<https://mlir.llvm.org/docs/Tutorials/QuickstartRewrites/>